

Overview of Policy gradient method

Subject: Policy gradient method

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi} [\nabla_{\theta} \log \pi_{\theta}(a | s) Q^{\pi}(s, a)]$$

1.

Policy gradient methods are a class of reinforcement learning algorithms and a sub-class of policy optimization methods. Unlike value-based methods which learn a value function to derive a policy, policy optimization methods directly learn a policy function π that selects actions without consulting a value function. For policy gradient to apply, the policy function π_{θ} is parameterized by a differentiable parameter θ .

2.

$\nabla_{\theta} J(\theta) = \mathbb{E}_{(s,a) \sim \pi_{\theta}} [\nabla_{\theta} \ln \pi_{\theta}(a | s) \cdot A^{\pi_{\theta}}(s, a)] = \nabla_{\theta} L(\theta, \theta)$ However, when $\theta \neq \theta_i$, this is not necessarily true. Thus it is a "surrogate" of the real objective. As with natural policy gradient, for small policy updates, TRPO approximates the surrogate advantage and KL divergence using Taylor expansions around θ_i : $L(\theta, \theta_i) \approx g^T(\theta - \theta_i)$, $D_{KL}(\pi_{\theta} \parallel \pi_{\theta_i}) \approx \frac{1}{2}(\theta - \theta_i)^T H(\theta - \theta_i)$, where:

3.

A further improvement is proximal policy optimization (PPO), which avoids even computing $F(\theta)$ and $F(\theta) - 1$ via a first-order approximation using clipped probability ratios. Specifically, instead of maximizing the surrogate advantage $\max_{\theta} L(\theta, \theta_t) = \mathbb{E}_{(s,a) \sim \pi_{\theta_t}} [\frac{\pi_{\theta}(a | s)}{\pi_{\theta_t}(a | s)} A^{\pi_{\theta_t}}(s, a)]$ under a KL divergence constraint, it directly inserts the constraint into the surrogate advantage: $\max_{\theta} \mathbb{E}_{(s,a) \sim \pi_{\theta_t}} [\min(\frac{\pi_{\theta}(a | s)}{\pi_{\theta_t}(a | s)}, 1 + \epsilon) A^{\pi_{\theta_t}}(s, a) \text{ if } A^{\pi_{\theta_t}}(s, a) > 0, \min(\frac{\pi_{\theta}(a | s)}{\pi_{\theta_t}(a | s)}, 1 - \epsilon) A^{\pi_{\theta_t}}(s, a) \text{ if } A^{\pi_{\theta_t}}(s, a) < 0]$ and PPO maximizes the surrogate advantage by stochastic gradient descent, as usual. In words, gradient-ascending the new surrogate advantage function means that, at some state s, a , if the advantage is positive: $A^{\pi_{\theta_t}}(s, a) >$

$A^{\pi_{\theta_t}}(s,a) > 0$, then the gradient should direct θ towards the direction that increases the probability of performing action a under the state s . However, as soon as θ has changed so much that $\pi_{\theta}(a|s) \geq (1 + \epsilon) \pi_{\theta_t}(a|s)$, then the gradient should stop pointing it in that direction. And similarly if $A^{\pi_{\theta_t}}(s,a) < 0$. Thus, PPO avoids pushing the parameter update too hard, and avoids changing the policy too much. To be more precise, to update θ_t to θ_{t+1} requires multiple update steps on the same batch of data. It would initialize $\theta = \theta_t$, then repeatedly apply gradient descent (such as the Adam optimizer) to update θ until the surrogate advantage has stabilized. It would then assign θ_{t+1} to θ , and do it again. During this inner-loop, the first update to θ would not hit the $1 - \epsilon, 1 + \epsilon$ bounds, but as θ is updated further and further away from θ_t , it eventually starts hitting the bounds. For each such bound hit, the corresponding gradient becomes zero, and thus PPO avoid updating θ too far away from θ_t . This is important, because the surrogate loss assumes that the state-action pair s, a is sampled from what the agent would see if the agent runs the policy π_{θ_t} , but policy gradient should be on-policy. So, as θ changes, the surrogate loss becomes more and more off-policy. This is why keeping θ proximal to θ_t is necessary. If there is a reference policy π_{ref} that the trained policy should not diverge too far from, then additional KL divergence penalty can be added: $-\beta \mathbb{E}_{s,a \sim \pi_{\theta_t}} [\log \left(\frac{\pi_{\theta}(a|s)}{\pi_{\text{ref}}(a|s)} \right)]$ where β adjusts the strength of the penalty. This has been used in training reasoning language models with reinforcement learning from human feedback. The KL divergence penalty term can be estimated with lower variance using the equivalent form (see f-divergence for details): $-\beta \mathbb{E}_{s,a \sim \pi_{\theta_t}} [\log \left(\frac{\pi_{\theta}(a|s)}{\pi_{\text{ref}}(a|s)} \right) + \pi_{\text{ref}}(a|s) \pi_{\theta}(a|s) - 1]$